

AulaFácil

Colegio Pedro de Gante

Manual técnico

Arquitectura, seguridad y mantenimiento

Versión 2.0 · Septiembre 2026

1. Qué es AulaFácil

AulaFácil es una aplicación web que sustituye el trabajo administrativo repetitivo de un maestro: aplicar exámenes, pasar lista, registrar participación y calcular calificaciones finales.

El principio de diseño que ordena todo el sistema es que AulaFácil no guarda la actividad académica, solo el marcador. El examen se contesta en la plataforma, pero los ejercicios de clase, las exposiciones y los trabajos ocurren fuera; el sistema únicamente registra quién participó, cuánto vale y cuándo.

2. Arquitectura

Son dos mitades que se comunican por HTTPS:

Capa	Tecnología	Dónde vive
Frontend	HTML + JavaScript puro, sin framework	GitHub Pages (archivos estáticos)
Estilos	Tailwind CSS compilado a un archivo	En el repositorio, generado
Base de datos	PostgreSQL con Row Level Security	Supabase (nube)
Autenticación	Supabase Auth	Supabase (nube)
Lógica sensible	Edge Functions en Deno/TypeScript	Supabase (nube)
Archivos	Supabase Storage (fotos de perfil)	Supabase (nube)

No hay servidor propio que mantener ni proceso que reiniciar. El frontend es un conjunto de archivos estáticos: cualquier servidor web que los sirva por HTTPS funciona.

3. Modelo de datos

La base tiene 24 tablas. Estas son las que sostienen el modelo:

Tabla	Qué guarda
profesores / alumnos	Perfiles, ligados uno a uno con los usuarios de Auth
grupos / grupo_alumnos	Clases y la inscripción de alumnos. Un alumno puede estar en varios grupos
periodos	Parciales del ciclo escolar, por grupo
exámenes / preguntas / opciones	Los exámenes y su contenido
intentos / respuestas	Lo que contestó cada alumno y su calificación
asistencias	Pase de lista. Estados: presente, retardo, falta, justificada
tareas / calificaciones_tareas	Tareas con peso y su calificación
participaciones / cortes_participacion	Modo Fichas y su sistema de corte

Tabla	Qué guarda
sesiones / actividades_sesion / participaciones_sesion	Modo Clase en vivo
sesiones_secreto	El secreto del QR rotativo, aislado a propósito
rubros_evaluacion / calificaciones_rubro	Rubros personalizados del maestro
ajustes_calificacion	Correcciones manuales del promedio, con motivo y autor

Decisiones de modelado que conviene entender

- Los puntos de participación se copian a la fila en el momento de aprobarse, no se leen en vivo desde la actividad. Si el maestro corrige el valor después, no se recalculan calificaciones ya emitidas.
- El resumen de una sesión (presentes, participaciones, puntos) se congela al cerrar la clase. Sirve como evidencia estable de lo que ocurrió ese día.
- El corte de participación en modo Fichas también se guarda como foto fija, no se recalcula.
- El secreto del QR vive en su propia tabla y no como columna de sesiones, porque Row Level Security en PostgreSQL opera por fila y no por columna: si fuera columna, un alumno que puede ver la sesión podría leer el secreto.
- Un ajuste manual de calificación no sobrescribe el cálculo: se guarda en su propia tabla con motivo y autor, y la interfaz muestra ambos valores. La trazabilidad importa más que la simplicidad del dato.
- Una reposición no reabre la sesión cerrada ni edita la participación original: se inserta como fila aparte en participaciones_sesion, marcada con origen, motivo_reposicion, porcentaje_reposicion y resuelto_en. El registro del día sigue siendo evidencia fiel.

Identidad del alumno: por maestro, no global

El diseño original hacía la matrícula única en todo el sistema, asumiendo que sería la del colegio. En la práctica eso no funciona: AulaFácil no es el sistema de control escolar, y los maestros no tienen acceso a las matrículas oficiales. Dos maestros que daban de alta al mismo alumno chocaban sin saber por qué.

Hoy alumnos.profesor_id marca al dueño, y la unicidad de la matrícula está limitada por maestro mediante un índice único parcial que además solo aplica a los alumnos activos. Un mismo estudiante puede existir varias veces, una por maestro, cada una con su propia credencial.

Como auth.users sí exige correo único globalmente, crear-alumnos prueba sufijos hasta encontrar uno libre y guarda el resultado en alumnos.correo_login, que es lo que la interfaz le muestra al maestro para que se lo dé al alumno.

Efecto lateral deseable: un alumno tiene credenciales distintas en cada materia, así que una contraseña prestada solo compromete esa clase. El costo es que la misma persona aparece como varias filas; es aceptable porque el sistema es el control del maestro, no el del colegio.

La sesión guardada en el teléfono

examen.js y panel-alumno.js toman la sesión que el navegador tenga guardada. Si es de otra cuenta — otra materia, otro maestro, o el propio tablero del maestro en un aparato compartido— la aplicación cargaba con esa identidad sin decirlo, RLS rechazaba, y el alumno veía "no perteneces al grupo" sin saber por qué ni cómo salirse. El 23 de septiembre eso produjo 64 respuestas 404 contra un solo intento de login fallido.

Ambas pantallas ahora muestran con qué cuenta se entró y ofrecen un botón para cerrar sesión y cambiarla. En el panel del alumno, además, escanear una clase que no le corresponde ya no dice "No hay clase activa" —que era falso— sino que la clase no es de esa cuenta.

Reinscribir a un alumno de otro ciclo

buscar-alumno.js es un buscador compartido por la pantalla de crear grupo y la de alumnos del grupo. Busca entre los alumnos del maestro por nombre o matrícula, incluidos los egresados, y al elegir uno manda su alumno_id exacto a crear-alumnos en lugar de nombre y matrícula. Así no hay forma de duplicar a nadie por accidente.

Si el alumno venía egresado, crear-alumnos lo reactiva: comprueba que su matrícula no la haya tomado otro alumno activo mientras tanto, y le devuelve su correo de entrada, que el egreso había renombrado a local.egresado-ciclo@dominio. Sin ese paso el alumno recuperaba su expediente pero se quedaba sin poder entrar.

Egreso: liberar matrículas sin borrar historia

Las columnas activo y ciclo_egreso de alumnos implementan el ciclo de vida. Al egresar, la Edge Function egresar-alumnos marca activo en falso, registra el ciclo y libera el correo de Auth, de modo que el índice único parcial deja de considerar esa matrícula y el número queda disponible otra vez.

Ningún dato académico se borra: intentos, respuestas, asistencias, participaciones y calificaciones siguen apuntando al mismo identificador y se consultan desde los grupos archivados. Solo se exige que el alumno ya no esté en ningún grupo activo, para evitar egresar a alguien que sigue en clase.

4. Seguridad

El supuesto de partida es que el alumno puede abrir las herramientas de desarrollo del navegador y llamar a la API directamente. Todo lo que importa está protegido del lado del servidor.

Row Level Security

Todas las tablas tienen RLS activa. Las reglas se apoyan en dos funciones auxiliares, es_profesor_del_grupo y alumno_en_grupo, que resuelven la pertenencia sin exponer datos de terceros.

Actor	Puede
Alumno	Leer solo lo suyo: sus intentos, sus participaciones, su asistencia

Actor	Puede
Alumno	NO puede escribir participaciones, ni crear sesiones, ni leer el secreto del QR
Maestro	Todo, pero únicamente dentro de los grupos de los que es dueño
Maestro ajeno	Nada de los grupos de otro maestro

Esto no es una suposición: se verificó suplantando las tres identidades contra la base real y comprobando que cada intento de abuso quedara bloqueado. Un alumno intentando insertarse 40 puntos recibe un error de política de seguridad.

Edge Functions

Concentran todo lo que no puede confiarse al navegador. Usan la llave de servicio, que nunca sale del servidor.

Son nueve funciones desplegadas:

Función	Responsabilidad
crear-alumnos	Alta en lote. Resuelve la matrícula contra los alumnos activos de ese maestro y negocia un correo libre
iniciar-examen	Entrega el examen sin respuestas correctas y ancla el cronómetro al reloj del servidor
enviar-respuestas	Califica en el servidor. Es idempotente, para tolerar reintentos sin duplicar
registrar-advertencia	Lleva la cuenta de avisos anti-copia del lado del servidor
validar-qr	Canjea el código del QR por un pase de entrada firmado. Es la única PÚBLICA: corre antes de que el alumno inicie sesión
registrar-presencia	Valida el pase (o el código) y marca asistencia, como presente o retardo según la tolerancia de la sesión
reclamar-participacion	Registra el reclamo del alumno como pendiente y sin puntos
cerrar-actividad	Convierte en puntos la lista definitiva que aprobó el maestro
egresar-alumnos	Marca el egreso y libera el correo de Auth para reciclar la matrícula

Consecuencia práctica: el alumno nunca recibe las respuestas correctas de un examen, no puede escribir su propia calificación y no puede otorgarse puntos de participación.

El QR rotativo

El código del QR se recalcula cada 20 segundos. Contiene una firma HMAC-SHA256 del identificador de la sesión y la ventana de tiempo, calculada en el navegador del maestro con el secreto que solo él puede leer. El servidor revalida aceptando la ventana actual y una de margen a cada lado, es decir unos 60 segundos de vida útil.

Esto resuelve el problema de que un alumno fotografíe el código y se lo mande a un compañero que está en su casa: para cuando llega, ya expiró.

El pase de entrada

Los 60 segundos de vida del código chocaban con la realidad: el alumno que llegaba sin sesión iniciada gastaba más que eso tecleando matrícula y contraseña, y su petición llegaba con el código ya vencido. El 18 de septiembre se rechazaron así 85 de 89 registros de asistencia, y como reclamar-participacion exige asistencia previa, los reclamos cayeron en cascada.

La solución fue separar las dos cosas. La función `validar-qr` es pública —se despliega con `--no-verify-jwt`— y corre en cuanto el teléfono abre el enlace, antes de cualquier login, que es cuando el código todavía está fresco. Si es válido devuelve un pase firmado con HMAC sobre el secreto de la sesión, vigente diez minutos. Al terminar de iniciar sesión, `registrar-presencia` cobra ese pase en lugar del código.

El pase no se guarda en ninguna tabla: se verifica recalculando la firma. Escanear sigue siendo obligatorio, porque sin haber apuntado la cámara a la pantalla nunca se obtiene uno.

Si `validar-qr` se vuelve a crear desde cero, hay que desplegarla con `--no-verify-jwt`. Si queda pidiendo sesión iniciada, el pase deja de funcionar y se regresa al problema original.

Por qué los códigos HTTP están diferenciados

Un 403 no se distingue de otro en los registros del servidor, y eso volvía imposible saber si los rechazos eran códigos vencidos, firmas inválidas o clases ya cerradas. `validar-qr` y `registrar-presencia` responden con códigos distintos a propósito:

Código	Motivo
409	La clase ya no está activa
410	El código o el pase ya vencieron
422	La firma no cuadra: código de otra clase, alterado o inventado
403	El alumno no pertenece a ese grupo

`iniciar-examen` aplica el mismo criterio: 404 si el enlace no corresponde a ningún examen, 409 si existe pero está en borrador o cerrado, y 403 si la cuenta con la que se entró no está en ese grupo. Además devuelve de quién es la sesión, para que la pantalla pueda decirlo.

Lo que protege el examen

El navegador no puede impedir una captura de pantalla ni una foto tomada con otro teléfono. En vez de fingir que sí, el sistema apuesta por dejar rastro y por quitar el incentivo:

- Marca de agua: sobre la pantalla del examen se dibuja, muy tenue, el nombre y la matrícula del alumno. Es un SVG repetido como fondo, con pointer-events en none para no estorbar. Cualquier captura o foto sale identificada.
- Barajado estable: `exámenes.barajar_preguntas` y `barajar_opciones`. El orden sale de una semilla derivada del id del intento (FNV-1a más mulberry32 más Fisher-Yates), así que a cada alumno le toca uno distinto pero siempre el mismo si recarga. No se guarda nada: se recalcula igual cada vez. Verdadero o falso no se baraja.

- Bitácora de copiado: los eventos copy, cut y la tecla ImprPant se anotan con conto en false. No suman advertencia ni bloquean; se muestran al maestro en Resultados, separados de las salidas de pantalla para que no inflen esa cuenta.
- Salidas de pantalla: se miden al regresar, restando Date.now(), no con un setTimeout programado al salir. El navegador estrangula los temporizadores en segundo plano y en iOS suspende la página entera, así que un temporizador nunca llegaba a ejecutarse.

El teclado en pantalla y la pantalla completa

En iPad, abrir el teclado saca a la página de pantalla completa. Eso disparaba fullscreenchange y se contaba como salida: con MAX_ADVERTENCIAS en 1, el alumno tocaba el campo de una pregunta de completar y se le entregaba el examen solo.

La detección ahora consulta document.activeElement: si un campo de texto tiene el foco, el teclado está arriba y no es una salida. Además se vuelve a pantalla completa en el siguiente toque, porque si no el alumno quedaría fuera para siempre y cualquier evento posterior contaría. Y en un aparato donde el teclado ya la tumbó, estar fuera de pantalla completa deja de bastar por sí solo: se exige también haber perdido el foco. En escritorio no cambia nada.

En iPhone requestFullscreen falla y usaFullscreen queda en false, así que toda esa rama está muerta ahí. Probar este comportamiento en iPhone no prueba nada: hay que hacerlo en iPad.

5. Resistencia a la red

La red del colegio falla con frecuencia y eso rompía los exámenes. Las medidas implementadas:

- Un aviso antes de bloquear. Salir del examen genera advertencia; a la segunda se cierra y se entrega.
- Las salidas que coinciden con un cambio de conectividad no cuentan, porque son la alerta de wifi del sistema tapando el navegador. Estar sin internet no da inmunidad: la advertencia se cuenta en el dispositivo y se reconcilia con el servidor al entregar.
- Las respuestas se guardan en el dispositivo a cada cambio. Recargar, quedarse sin batería o cerrar la pestaña ya no las pierde.
- La entrega se reintenta con esperas crecientes hasta que el servidor confirma. La pantalla de "entregado" solo aparece cuando de verdad se guardó.
- El cronómetro se ancla a la hora de inicio registrada en el servidor, así que recargar no regala tiempo.
- Un service worker guarda el cascarón de la aplicación para que abra sin red. Las llamadas a Supabase nunca se cachean.
- Las pantallas del alumno no dependen de ningún CDN externo: Tailwind viene compilado y el SDK de Supabase vive en el repositorio.

Detección en iOS: Safari congela los temporizadores de una página en segundo plano, de modo que un setTimeout programado al salir puede no ejecutarse nunca. Por eso la salida se mide al regresar, restando con reloj de pared. Si alguien modifica esta lógica, debe conservar ese enfoque o la detección dejará de funcionar en iPhone.

6. Versionado del esquema

El esquema no se administra a mano desde el panel de Supabase: cada cambio es una migración con nombre y fecha. A la fecha de este documento hay 32 migraciones, desde la creación del esquema hasta la tolerancia de retardo opcional, y viven en `supabase/migrations` dentro del repositorio.

Esto es lo que hace reproducible el sistema. Con el repositorio y una cuenta de Supabase vacía se puede levantar una instancia idéntica sin depender del proyecto actual:

1. `npx supabase link --project-ref <identificador del proyecto nuevo>`
2. `npx supabase db push`, que aplica las migraciones en orden
3. `npx supabase functions deploy <nombre>`, una por cada Edge Function

Si se toca la base directamente desde el panel, hay que bajar el cambio con `npx supabase db pull` para que quede registrado. Un cambio aplicado solo en producción existe únicamente ahí y se pierde en cualquier migración futura.

Despliegue

El sitio se sirve hoy desde GitHub Pages. Para moverlo a otro servidor basta copiar el contenido del repositorio a la raíz del sitio: las rutas del proyecto son relativas y funcionan igual en la raíz que en una subcarpeta.

El servidor debe servir por HTTPS con certificado válido. El service worker solo funciona en contexto seguro, y la cámara para escanear el QR también lo exige. Sin HTTPS se pierde el modo offline y la clase en vivo.

7. Mantenimiento

Al modificar archivos del examen

Hay que subir el número de VERSION en `sw.js`. Si no se hace, los dispositivos que ya abrieron la aplicación siguen usando la versión guardada. Para que un dispositivo tome la versión nueva, el botón de la esquina superior derecha cambia a color rojo y dice "Actualizar".

Riesgos operativos conocidos

- El plan gratuito de Supabase suspende los proyectos sin actividad después de aproximadamente una semana. En periodos vacacionales la plataforma puede amanecer caída y requiere que alguien la reactive desde el panel.
- Reactivar un intento de examen después de la hora de cierre no funciona: la función iniciar-examen rechaza crear intentos nuevos una vez pasada la fecha límite.
- La clase "rounded-DEFAULT" aparece en el HTML pero Tailwind no genera esa clase. Las esquinas redondeadas que se pretendían nunca se aplicaron. Corregirlo cambiaría el aspecto de toda la aplicación.
- La barra lateral del panel principal tiene etiquetas en inglés mezcladas con el resto en español.

- El borrado definitivo de un alumno no se hace desde la aplicación: requiere eliminar la cuenta en `auth.users`, y el borrado cascadea a todo su historial. La operación normal es el egreso, que conserva los datos.
- `auth.users` es el PADRE, no el hijo: `alumnos.id` y `profesores.id` apuntan a él con `ON DELETE CASCADE`, y grupos cuelga de profesores igual. Borrar la cuenta de un maestro se lleva sus grupos, alumnos, exámenes y calificaciones sin preguntar. Cualquier borrado en `auth.users` debe llevar guardas que excluyan a quien tenga ficha de maestro o de alumno.
- Quitar a un alumno de un grupo no borra su ficha ni su cuenta de acceso. Se acumulan invisibles: ya se limpiaron a mano dos veces, 77 cuentas la primera y 37 la segunda.
- El servicio de correo integrado de Supabase entrega dos mensajes por hora. Con varios maestros registrándose el mismo día, del tercero en adelante el correo de confirmación no llega y no hay aviso de por qué. Se resuelve configurando SMTP propio, que sube el límite a treinta.
- Dieciocho de las veintidós pantallas cargan Tailwind desde `cdn.tailwindcss.com` en vez del `app.css` compilado que el repositorio ya trae. Si la red del colegio bloquea ese dominio, esas pantallas se ven sin estilos. Además, en ese build la clase `flex` va después de `hidden`, así que `hidden` pierde: los diálogos de esas páginas se abren y cierran con estilo en línea, no con la clase.
- El service worker sirve el código con red primero y cae a la copia guardada si la red tarda más de 3 segundos (`LIMITE_MS`). En un salón con wifi saturado eso puede entregar JavaScript viejo.
- El código del QR se genera con el reloj de la computadora del maestro pero lo valida el reloj del servidor. Si esa computadora se desfasa más de unos 20 segundos, la asistencia deja de funcionar por completo y sin mensaje que lo explique.

Auditoría de seguridad

El analizador de Supabase no reporta ningún error sobre el proyecto: no hay tablas sin RLS ni políticas permisivas. Las advertencias abiertas son menores: las funciones auxiliares `es_profesor_del_grupo` y `alumno_en_grupo` quedan expuestas como RPC, aunque solo devuelven un booleano sobre el propio usuario, y la verificación de contraseñas filtradas de Supabase Auth está desactivada.

8. Accesos a transferir

Para que la institución quede como dueña de la operación se deben transferir:

Qué	Cómo
Proyecto de Supabase	Transferirlo a la organización del colegio desde Project Settings. Conserva usuarios, datos y contraseñas, y el identificador no cambia, por lo que no hay que tocar código
Repositorio de GitHub	Transferirlo o crear una copia en la cuenta institucional. Cambia la dirección pública del sitio
Dominio y hosting	Según lo que decida la institución

La transferencia del proyecto de Supabase es preferible a crear uno nuevo y migrar. Recrear los usuarios generaría identificadores distintos y dejaría huérfano todo el historial de calificaciones, asistencia y participación, que apunta a los identificadores actuales.